How Squishy Jiggly Pals Works

Nov 6, 2026 · @pd

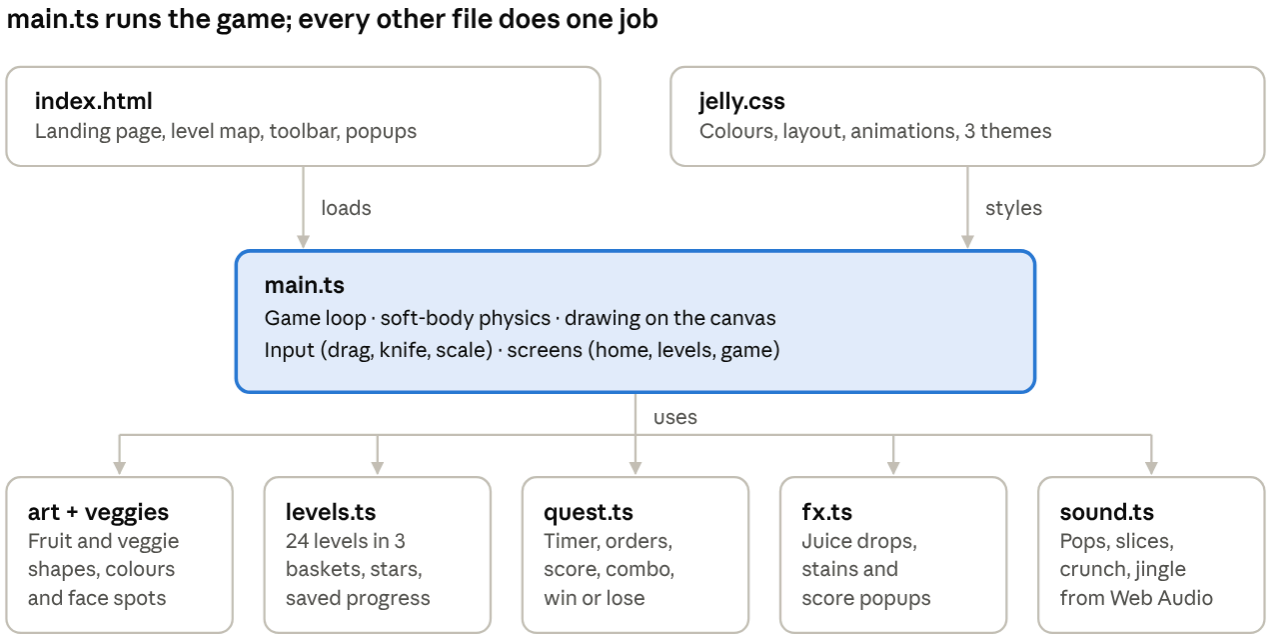
Overview

Squishy Jiggly Pals is a browser game made with plain TypeScript, HTML and CSS: no game engine and no framework. Every jelly is drawn in code on an HTML5 Canvas, wobbles with home-made soft-body physics, and can be dragged, sliced, eaten, scaled and thrown.

Part	Technology	Why
Language	TypeScript	JavaScript with types, so mistakes show up before the game runs
Page and buttons	HTML + CSS	Landing page, level map, toolbar, popups and their animations
Build tool	Vite	Fast dev server with instant reload, and one command to build the site
Drawing	HTML5 Canvas 2D	Draws shapes, gradients, shine, shadows and faces 60 times a second
Physics	Custom code (shape matching)	Makes jellies wobble and collide without a physics library
Sound	Web Audio API	Creates every sound from code, so there are no audio files
Saving	localStorage	Keeps stars, best scores and the basket choice in the browser

The whole game builds to about 50 KB of JavaScript.

Project structure



project files · what each one does

The browser opens index.html, which loads main.ts. main.ts runs the game and calls the five helper files below it whenever it needs a design, a level, a timer, a splash or a sound.

Step 1: Set up the project and the game loop

Start with a Vite + TypeScript project, put one full-screen `<canvas>` in the page, and run a loop that updates and redraws the scene about 60 times a second.

1. Create the project and install the two tools:

```
npm create vite@latest squishy-jiggly-pals -- --template vanilla-ts
cd squishy-jiggly-pals
npm install
npm run dev
```

2. In `index.html`, add the canvas plus the HTML for buttons and screens. The buttons sit on top of the canvas with CSS `position: fixed`.
3. In `main.ts`, size the canvas to the window and get its 2D drawing context.
4. Run the game loop with `requestAnimationFrame`. Physics runs in fixed 1/60-second steps, so the jellies move the same on fast and slow screens:

```
function frame(now: number) {
  acc += Math.min(50, now - last)   // time since last frame
  last = now
  while (acc >= 1000 / 60) {        // physics in fixed steps
    step()
    acc -= 1000 / 60
  }
  ctx.clearRect(0, 0, W, H)          // wipe the screen
  for (const b of bodies) drawBody(b) // draw every jelly
  requestAnimationFrame(frame)       // ask for the next frame
}
requestAnimationFrame(frame)
```

Every feature below plugs into this loop: physics goes in `step()`, pictures go in the drawing part.

Step 2: Draw a cute jelly on the canvas

A jelly is a ring of about 44 points around its edge, plus a picture (its art) drawn inside that ring. Drawing it is five layers, back to front:

1. **Shadow**: fill the outline with `ctx.shadowBlur` and `shadowOffsetY` set, so it looks lifted off the table.
2. **Art, clipped**: trace the outline as a smooth curve, call `ctx.clip()`, then draw the fruit art (kiwi seeds, lemon segments, mushroom spots). Anything outside the wobbly outline is hidden.
3. **Volume**: a radial gradient, light at the top-left and darker at the bottom-right, makes it look round.
4. **Shine**: a white curved stroke along the top-left edge, plus one small white dot, makes it glossy like jelly.
5. **Extras**: stems and leaves (carrot tops, tomato star, beetroot leaves) drawn outside the clip.

The smooth outline comes from drawing a curve through the middle of each pair of neighbouring points:

```
ctx.moveTo(midX(last, first), midY(last, first))
for (let i = 0; i < n; i++) {
  const j = (i + 1) % n
  ctx.quadraticCurveTo(px[i], py[i], (px[i] + px[j]) / 2, (py[i] + py[j]) /
2)
}
```

Each fruit or veggie is a small object with an `outline()` (its shape) and an `art(ctx)` function (its picture), so adding a new one means writing one object in `art.ts` or `veggies.ts`.

Step 3: Make it wobble (soft-body physics)

The wobble comes from a technique called **shape matching**: every point remembers where it should sit, and each frame it is gently pulled back toward that spot. Because the pull is gentle, the points overshoot and bounce, and that bounce is the jiggle.

Each physics step does this for every jelly:

1. **Move**: each point moves by its speed (a little slowed down so it settles).
2. **Find the centre**: average all the points.
3. **Find the rotation**: work out how much the jelly has turned, so it can spin naturally.
4. **Find the goal**: take the original shape, turn it by that rotation and place it on the centre. Each point now has a goal spot.
5. **Pull**: move each point a small fraction of the way to its goal.
6. **Update speed**: new speed = new position minus old position.

```
// step 5: pull each point toward its goal
px[i] += (goalX - px[i]) * stiffness
py[i] += (goalY - py[i]) * stiffness
```

`stiffness` is what the **Hard** and **Soft** buttons change: 0.24 makes a firm jelly, 0.06 a very wobbly one. The art inside also squashes and stretches, because the game measures how the points have deformed and applies the same stretch to the picture.

Step 4: Dragging, bumping and gravity

These are small extra rules added inside the same physics step.

- **Dragging**: on `pointerdown`, find the jelly under the finger with a point-in-polygon test. Points near the finger get a strong pull toward it and far points get almost none, so the jelly stretches like real jelly. Letting go makes it bounce back. A fast release counts as a fling.
- **Bumping**: if a point of one jelly ends up inside another jelly, push it out to the nearest edge, and push that edge back by the same amount. A small push between the two centres stops deep overlaps getting stuck.
- **Walls and floor**: any point past the screen edge is moved back inside.

- **Gravity**: when the Gravity button is on, every point gets a small downward nudge each step, so the jellies fall and pile up. A little floor friction stops them sliding forever.
- **Shake**: adds a random push to every point at once.
- **Scale**: changes the size the shape-matching goal uses, and a spring makes the size bounce a little as it changes.

Step 5: Slice a jelly with the knife

A slice cuts the jelly's ring of points into two smaller rings, and each becomes a new jelly that keeps the same picture.

1. While the knife is down, each mouse movement is a short line segment. Check it against every edge of every jelly (segment intersection).
2. When the swipe has crossed the same jelly's outline **twice**, you have an entry point and an exit point.
3. Walk around the ring: points from entry to exit form piece A, the rest form piece B. Add both cut points to each piece, plus a few extra points along the cut so the new flat side can wobble too.
4. Each piece remembers which part of the original picture it holds, so a sliced kiwi still shows its seeds in the right place.
5. Push the two pieces apart a little, spray juice, play the slice sound, and count it for the current order.

Pieces smaller than a minimum size are not cut, so a jelly can't be chopped into dust.

Step 6: Cute faces, juice and sound

Faces. Each design says where its face sits (a mushroom's is on the stem, a carrot's near the top). The face is drawn with the same squash and turn as the jelly, so it stretches when you pull.

- Eyes: two dark ovals with a white sparkle. They shift slightly toward the mouse, so the jellies watch you.
- Blink: a timer per jelly; every few seconds the eyes flatten for 9 frames.
- Mood: grabbed shows squeezed `> <` eyes and an open mouth; moving fast or just sliced shows big surprised eyes; otherwise a small "w" smile and pink cheeks.

Juice. Slicing or eating creates about 20 to 30 small coloured drops that fly out and fall, plus a soft stain on the table that fades over about 7 seconds. Score popups like "+100 Nice!" float up and fade. Each design has its own juice colour.

Sound. There are no audio files. Each sound is built with the Web Audio API from an oscillator (a tone) or a burst of random noise, with its pitch and volume changed over a fraction of a second:

```
const o = audio.createOscillator()
o.frequency.setValueAtTime(260, t)              // start low
o.frequency.exponentialRampToValueAtTime(560, t + 0.1) // slide up = "bloop"
```

Pop, grab, wobble, slice, crunch, whoosh, tick, win and lose are all made this way.

Step 7: Levels, Rush mode, baskets and saving

A level is just data: a list of orders, a time limit and which jellies appear. The round engine in `quest.ts` reads that data, so adding a level needs no new code.

```
{
  name: 'First Chop', emoji: '🥕', time: 60,
  pool: ['carrot', 'mushroom', 'tomato'],
  tasks: [t('slice', 'carrot', 'a Carrot'), t('slice', 'mushroom', 'a
Mushroom')],
}
```

How a round works:

1. The game sends an event each time you slice, eat, wobble, grow, shrink or fling a jelly.
2. The engine checks it against the current order ("Slice Mushrooms ×3") and counts progress.
3. A finished order gives 100 points, times the combo if you were quick. The next order appears.
4. Levels end when every order is served (win) or the timer hits zero (lose). Stars come from time left: 45% or more left gives 3 stars, 20% or more gives 2, otherwise 1.
5. **Rush mode** uses the same engine, but invents random orders forever and adds 5 seconds for each one you serve.

Baskets. Fruits, Veggies and Both each have their own list of 8 levels, their own jellies on the table and their own chips at the top.

Saving. Progress is stored with `localStorage`, which keeps small text values in the browser even after it closes:

```
localStorage.setItem('jelly-orchard-levels-veggie', JSON.stringify([3, 2,
0]))
```

That line saves 3 stars on level 1 and 2 stars on level 2. A level unlocks when the one before it has at least 1 star.

Run, build and extend

The source is on GitHub at dulmini11/Squishy-Jiggly-Pals-Game.

```
git clone https://github.com/dulmini11/Squishy-Jiggly-Pals-Game.git
cd Squishy-Jiggly-Pals-Game
npm install
npm run dev      # play at http://localhost:5173
npm run build    # finished site goes into dist/
```

The `dist/` folder is plain HTML, CSS and JavaScript, so it can be hosted on any static host such as GitHub Pages, Netlify or Vercel.

To add	Where	What to write
A new fruit or veggie	`art.ts` or `veggies.ts`	An object with `outline()`, `art(ctx)`, a juice colour and a face spot
A new level	`levels.ts`	A name, time, pool of jellies and a list of orders
A new order type	`quest.ts` + `main.ts`	Send a new event from the game and match it in the engine
A new sound	`sound.ts`	A tone or noise with a pitch and volume slide